

Name: Nilanjan Ghosh

College: SRM Institute of Science and Technology

Course: Java FSE Digital Nurture

Week 1

Exercise 1: Setting Up JUnit

Aim

To set up JUnit in a Java Maven project and create a test class for performing unit testing.

Objective

The objective of this exercise is to configure JUnit in a Java project using Maven and verify the setup by creating and executing a sample test case.

Software Requirements

- Java Development Kit (JDK)
 - Apache Maven
 - Visual Studio Code / IntelliJ IDEA / Eclipse
 - JUnit 4.13.2
-

Procedure

Step 1: Create a New Maven Project

A new Maven project named **JUnitExercise1** was created using Maven Archetype.

Step 2: Add JUnit Dependency

The following dependency was added to the pom.xml file to include JUnit 4.13.2 in the project.

```
<dependency>

    <groupId>junit</groupId>

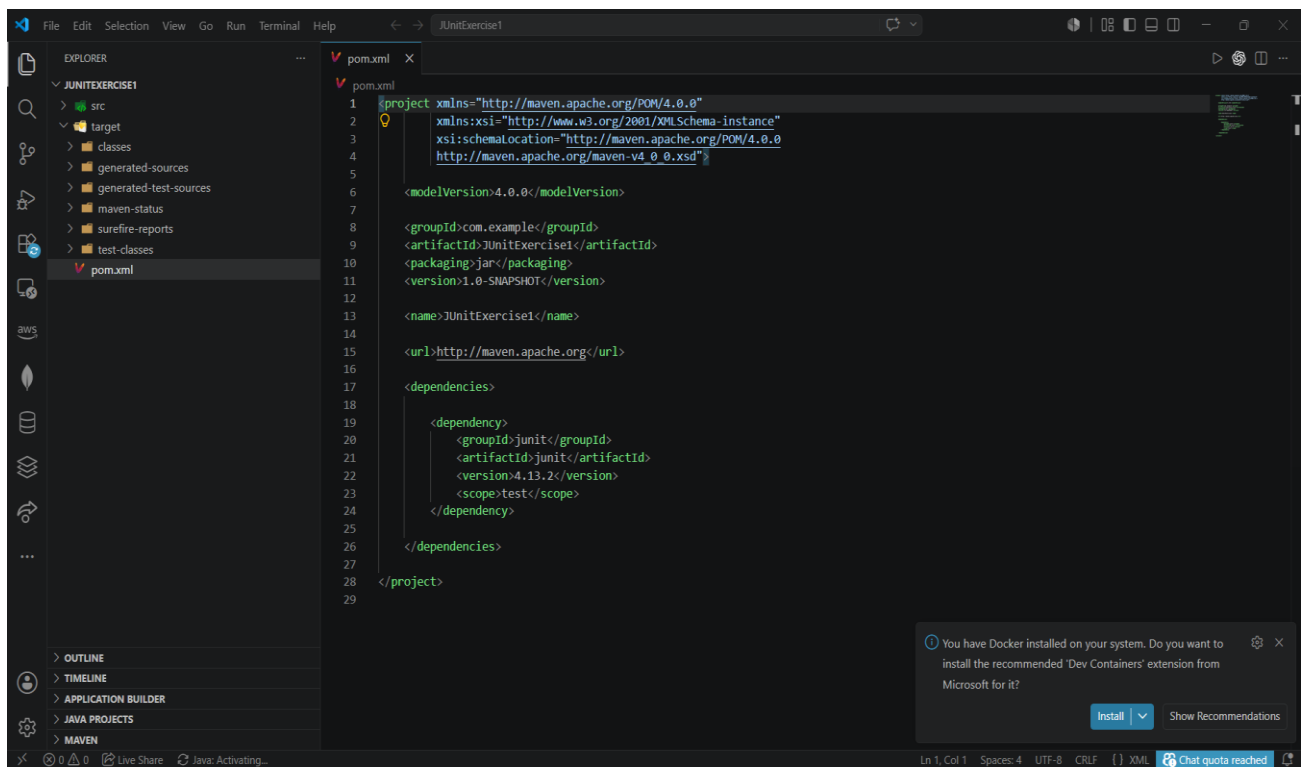
    <artifactId>junit</artifactId>

    <version>4.13.2</version>

    <scope>test</scope>

</dependency>
```

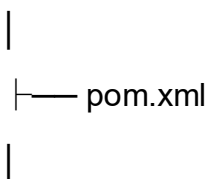
Screenshot 1:



Step 3: Project Structure

The Maven project structure consists of:

JUnitExercise1

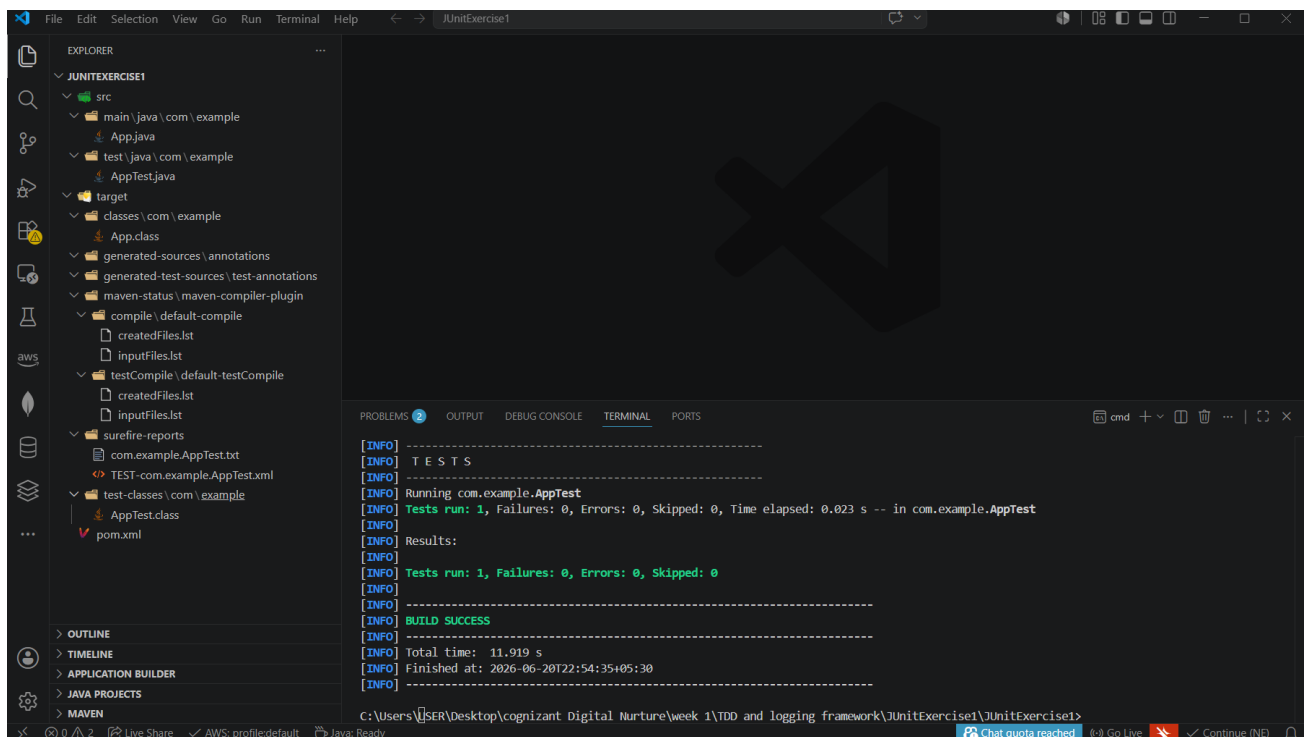


```

└─ src
    └─ main
        └─ java
            └─ com.example
                └─ App.java
    └─ test
        └─ java
            └─ com.example
                └─ AppTest.java

```

Screenshot 2:



Step 4: Create a Test Class

A test class named AppTest.java was created inside the src/test/java directory.

Example:

```
package com.example;
```

```
import org.junit.Test;
```

```
import static org.junit.Assert.*;
```

```
public class AppTest {
```

```
    @Test
```

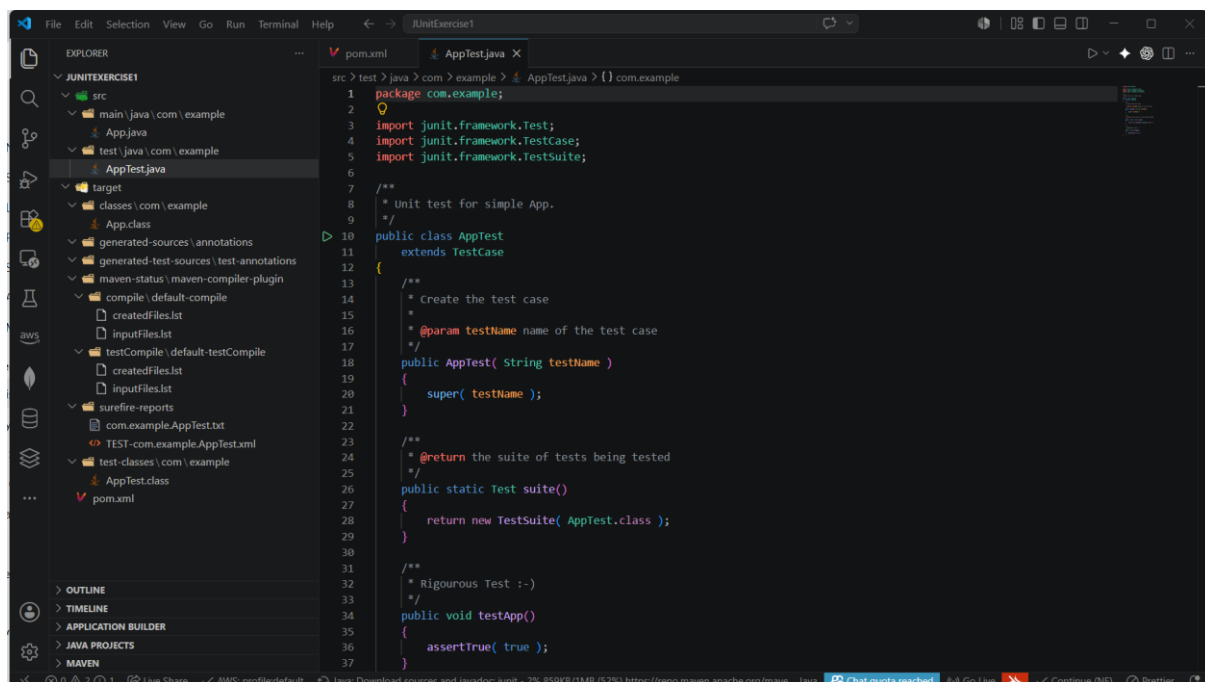
```
    public void testAddition() {
```

```
        assertEquals(5, 2 + 3);
```

```
    }
```

```
}
```

Screenshot 3:



Step 5: Execute Test Cases

The Maven test command was executed:

```
mvn test
```

The build was completed successfully and the test case passed.

Output

The test case executed successfully and Maven reported:

Tests run: 1

Failures: 0

Errors: 0

BUILD SUCCESS

Screenshot 4:



The screenshot shows a terminal window with the following output:

```
[INFO] -----  
[INFO] T E S T S  
[INFO] -----  
[INFO] Running com.example.AppTest  
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.023 s -- in com.example.AppTest  
[INFO] Results:  
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0  
[INFO] -----  
[INFO] BUILD SUCCESS  
[INFO] -----  
[INFO] Total time: 11.919 s  
[INFO] Finished at: 2026-06-20T22:54:35+05:30  
[INFO] -----  
c:\Users\USER\Desktop\cognizant Digital Nurture\week 1\TDD and logging framework\JUnitExercise1\JUnitExercise1>
```

At the bottom of the terminal, there is a status bar with the text "Chat quota reached" and a "Go Live" button.

Result

JUnit 4.13.2 was successfully configured in the Maven project. A test class was created and executed successfully using Maven, demonstrating the basic setup of JUnit for unit testing in Java applications.

Conclusion

This exercise provided practical experience in setting up JUnit within a Maven-based Java project. The successful execution of test cases confirmed that the testing framework was properly configured and ready for future unit testing activities.

Exercise 3: Assertions in JUnit

Aim

To understand and implement various JUnit assertions for validating test results in Java applications.

Objective

The objective of this exercise is to learn how different assertion methods provided by JUnit can be used to verify expected outcomes during unit testing.

Software Requirements

- Java Development Kit (JDK)
 - Apache Maven
 - Visual Studio Code
 - JUnit 4.13.2
-

Theory

Assertions are methods provided by JUnit that help verify whether the actual result matches the expected result. If an assertion fails, the test case is marked as failed.

Common JUnit Assertions include:

- assertEquals()
- assertTrue()
- assertFalse()
- assertNull()
- assertNotNull()

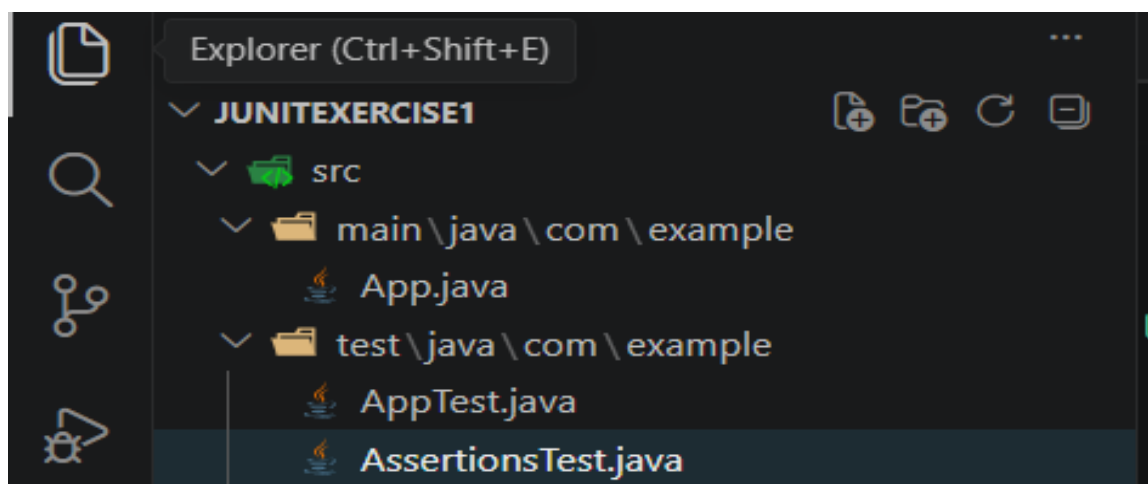
These assertions are used to validate program behavior and ensure software correctness.

Procedure

Step 1: Create a Test Class

A test class named AssertionsTest.java was created inside the src/test/java directory.

Screenshot 1:



Step 2: Implement Assertions

The test method contains different assertions to verify arithmetic operations, boolean conditions, and object references.

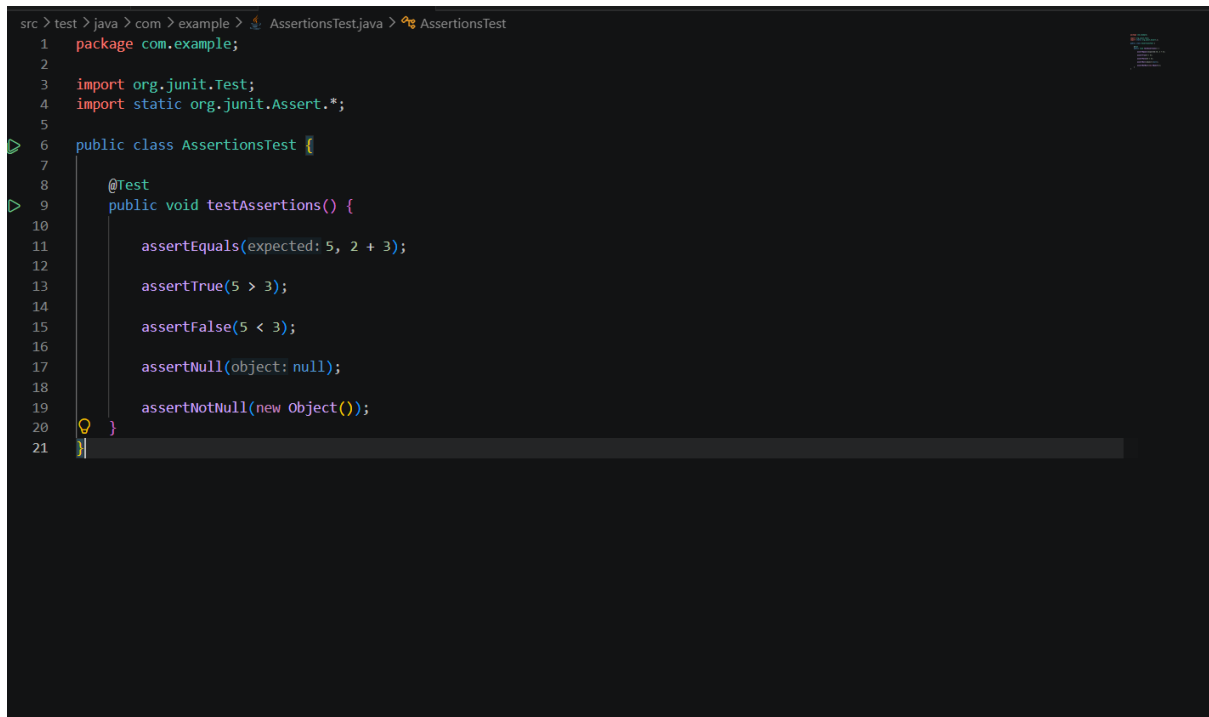
Assertions Used:

1. `assertEquals()`
2. `assertTrue()`

3. `assertFalse()`
4. `assertNull()`
5. `assertNotNull()`

Screenshot 2:

Insert screenshot showing the complete source code of `AssertionsTest.java`.



```
src > test > java > com > example > AssertionsTest.java > AssertionsTest
1 package com.example;
2
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6 public class AssertionsTest {
7
8     @Test
9     public void testAssertions() {
10
11         assertEquals(expected: 5, 2 + 3);
12
13         assertTrue(5 > 3);
14
15         assertFalse(5 < 3);
16
17         assertNull(object: null);
18
19         assertNotNull(new Object());
20     }
21 }
```

Step 3: Execute Test Cases

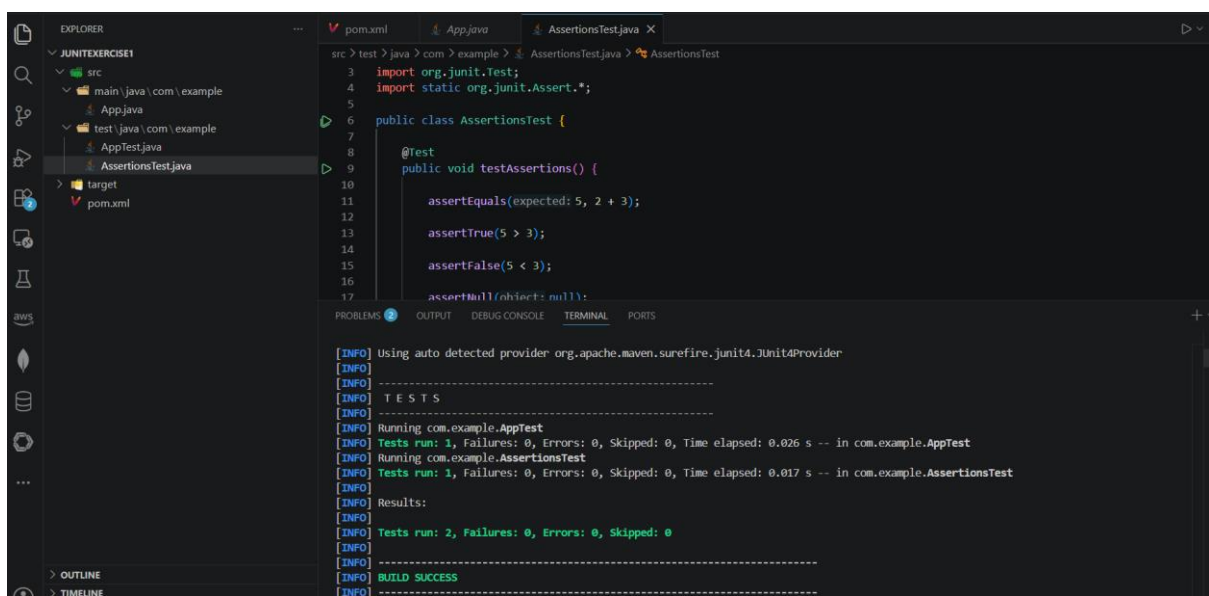
The Maven test command was executed:

`mvn test`

JUnit executed all assertions successfully.

Screenshot 3:

Insert screenshot showing the terminal command `mvn test`.



```
EXPLORER
src
main \ java \ com \ example
App.java
test \ java \ com \ example
AppTest.java
AssertionsTest.java
target
pom.xml

pom.xml App.java AssertionsTest.java
src > test > java > com > example > AssertionsTest.java > AssertionsTest
3 import org.junit.Test;
4 import static org.junit.Assert.*;
5
6 public class AssertionsTest {
7
8     @Test
9     public void testAssertions() {
10
11         assertEquals(expected: 5, 2 + 3);
12
13         assertTrue(5 > 3);
14
15         assertFalse(5 < 3);
16
17         assertNull(object: null);
18
19         assertNotNull(new Object());
20     }
21 }
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.026 s -- in com.example.AppTest
[INFO] Running com.example.AssertionsTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in com.example.AssertionsTest
[INFO] Results:
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Output

The test execution completed successfully.

Output:

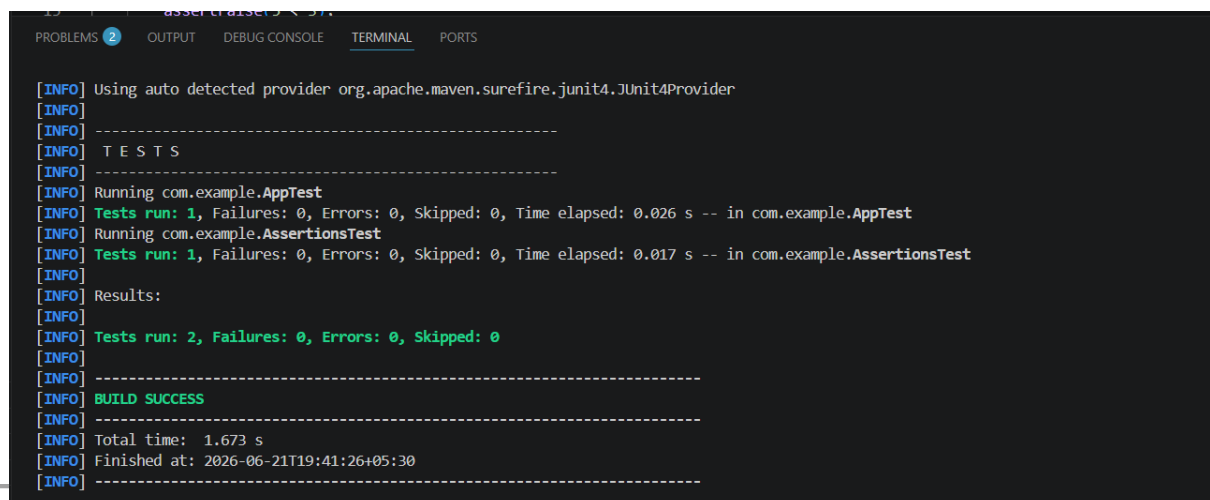
Tests run: 1

Failures: 0

Errors: 0

BUILD SUCCESS

Screenshot 4:

A screenshot of an IDE terminal window with a dark background. The terminal shows the output of a Maven test execution. It starts with an info message about the JUnit4 provider, followed by a separator line and the word 'TESTS'. Then, it shows two test classes being run: 'com.example.AppTest' and 'com.example.AssertionsTest'. Each test run is followed by a summary line: 'Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.026 s -- in com.example.AppTest' and 'Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in com.example.AssertionsTest'. After a 'Results:' section, a final summary line states 'Tests run: 2, Failures: 0, Errors: 0, Skipped: 0'. This is followed by another separator line and the message 'BUILD SUCCESS'. The terminal concludes with 'Total time: 1.673 s' and 'Finished at: 2026-06-21T19:41:26+05:30'. The IDE interface at the top shows tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS.

```
[INFO] Using auto detected provider org.apache.maven.surefire.junit4.JUnit4Provider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.026 s -- in com.example.AppTest
[INFO] Running com.example.AssertionsTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.017 s -- in com.example.AssertionsTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 1.673 s
[INFO] Finished at: 2026-06-21T19:41:26+05:30
[INFO]
[INFO] -----
```

Result

The JUnit assertions were successfully implemented and executed. All assertions passed, confirming the correctness of the test conditions.

Conclusion

This exercise demonstrated the use of various JUnit assertion methods for validating expected results. Assertions are an essential part of unit testing as they help verify application behavior and ensure software quality.